Università di Pisa
Corso di Laurea in Informatica

Il caso WannaCry: analisi statica con Ghidra e sviluppo sperimentale di uno script di detection

Candidato:
Stefano Raglianti

Relatore:
Prof. Vincenzo Gervasi

Anno Accademico 2024/2025

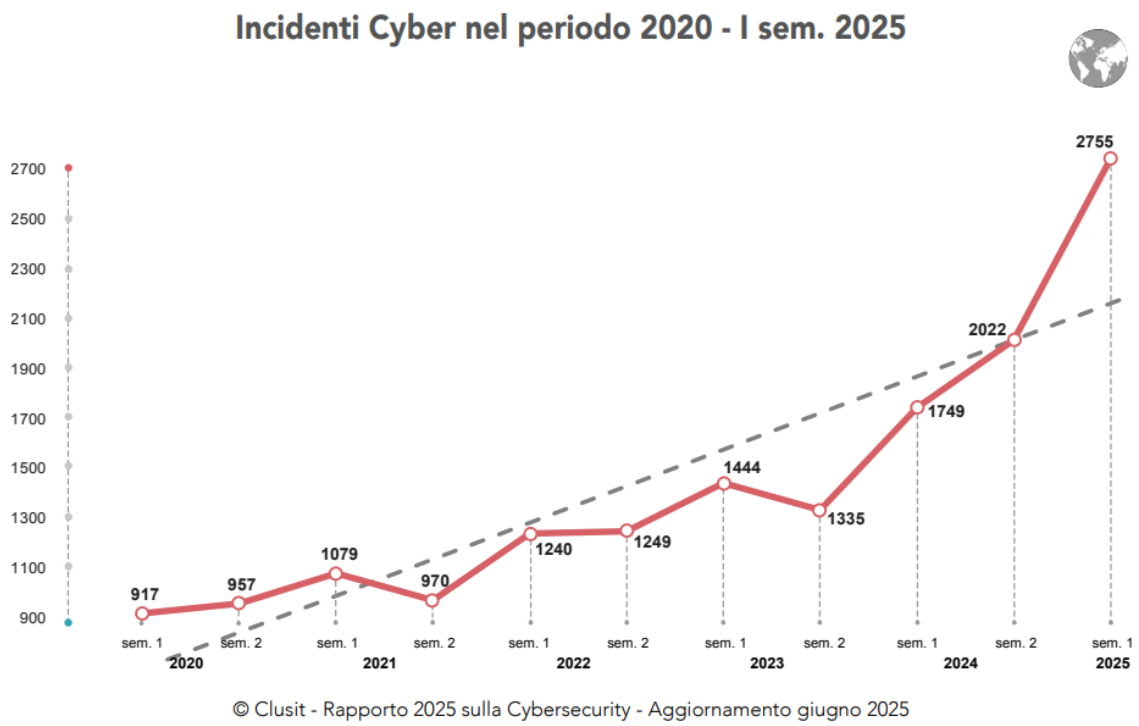
Il Contesto: La minaccia Ransomware

Trend preoccupante:

- Incidenti Cyber triplicati nell'ultimo quinquennio.
- L'87% degli incidenti è intenzionale (attacchi informatici).

La popolarità dei Ransomware

- L'83.5% degli attacchi a seguito di un'intrusione sono ransomware.
- Ogni attacco ransomware procura in media di 1,53 milioni di dollari alla vittima.



Il caso di studio: WannaCry

Natura della minaccia

- **Ransomware**: malware che cifra i file della vittima e chiede il pagamento di un riscatto per la chiave di decifratura.
- **Worm**: infetta altre macchine in maniera autonoma.

Workflow di WannaCry

- **Infezione e Backdoor:** Sfrutta l'exploit *EternalBlue* (vulnerabilità del protocollo SMBv1) per penetrare nel sistema e installa la backdoor *DoublePulsar*.
- **Il Dropper:** Si occupa di estrarre e decifrare le risorse malevole.
- **Il Payload:** Esegue la cifratura dei file, modifica il desktop e gestisce la persistenza.

L'Approccio e lo Strumento: Analisi Statica con Ghidra

La Metodologia: L'Analisi Statica

- **Cos'è:** L'analisi del codice senza la sua effettiva esecuzione.
- **Perché sceglierla:** Maggiore sicurezza e visione completa di ogni possibile percorso del malware.

Lo Strumento: Ghidra SRE Framework

- **Cos'è:** Piattaforma open-source di Software Reverse Engineering sviluppata dalla *National Security Agency* (NSA).
- **Disassembler & Decompiler:** Traduzione dal binario ad Assembly e a codice C ad alto livello, con supporto a una vasta gamma di architetture.
- **Il vantaggio principale:** estensibile con un supporto nativo allo scripting (Python e Java).

Interfaccia di Ghidra: Disassembler e Decompiler

CodeBrowser: Wannacry:/tasksche.exe

File Edit Analysis BSim Graph Navigation Search Select Tools Window Help

Program Trees

- tasksche.exe
  - Headers
  - .text
  - .rdata
  - .data
  - .rsrc
  - tdb

Program Tree

Symbol Tree

- Exports
  - entry
- Functions
  - entry

Filter: entry

Listing: tasksche.exe

```
        undefined        Stack[-0x74]:1local_74                    XREF[1]:     00407867(*)
        undefined4       Stack[-0x78]:4local_78                    XREF[4]:     0040788e(W),
                                                                                  00407897(W),
                                                                                  004078aa(W),
                                                                                  004078d6(W)
        undefined4       Stack[-0x7c]:4local_7c                    XREF[1]:     004078ff(W)
                                          $U20b00:4 = COPY EBP
                                          ESP = INT_SUB ESP, 4:4
                                          STORE ram(ESP), $U20b00:4
                         entry                                      XREF[2]:
004077ba 55              PUSH       EBP
                                          EBP = COPY ESP
004077bb 8b ec           MOV        EBP,ESP
                                          $U43f00:4 = COPY 0xffffffff:4
                                          ESP = INT_SUB ESP, 4:4
                                          STORE ram(ESP), $U43f00:4
004077bd 6a ff           PUSH       -0x1
                                          $U44100:4 = COPY 0x40d488:4
                                          ESP = INT_SUB ESP, 4:4
                                          STORE ram(ESP), $U44100:4
004077bf 68 88 d4        PUSH       DAT_0040d488                    = FFh
         40 00
                                          $U44100:4 = COPY 0x4076f4:4
                                          ESP = INT_SUB ESP, 4:4
                                          STORE ram(ESP), $U44100:4
004077c4 68 f4 76        PUSH       DAT_004076f4                    = FFh
         40 00
                                          $U9080:4 = INT_ADD FS_OFFSET, 0:4
```

Decompile: entry - (tasksche.exe)

```
1
2 /* WARNING: Globals starting with '_' overlap smaller
3
4 void entry(void)
5
6 {
7   undefined4 *puVar1;
8   byte *pbVar2;
9   char **local_74;
10  _startupinfo local_70;
11  int local_6c;
12  char **local_68;
13  int local_64;
14  _STARTUPINFOA local_60;
15  undefined1 *local_1c;
16  void *pvStack_14;
17  undefined *puStack_10;
18  undefined *puStack_c;
19  undefined4 local_8;
20
21  puStack_c = &DAT_0040d488;
22  puStack_10 = &DAT_004076f4;
23  pvStack_14 = ExceptionList;
24  local_1c = &stack0xffffff78;
25  local_8 = 0;
26  ExceptionList = &pvStack_14;
27  __set_app_type(2);
```

Decompile: entry | Bytes: tasksche.exe

Console - Scripting

Function Call Trees: entry - (tasksche.exe)

Incoming | Outgoing

004077bf | entry | PUSH 0x40d488

L'Astrazione del Binario: Il P-Code

Cos'è il P-Code?

- Linguaggio di *Rappresentazione Intermedia* (IR) nativo di Ghidra.
- Traduce le istruzioni Assembly in micro-operazioni logiche, astraendo dall'architettura hardware (es. x86, ARM).

Strutture Fondamentali

- **Varnode:** è la generalizzazione di qualsiasi valore manipolato dal programma. È composto dalla seguente tripla:

$$V = \{\text{Space}, \text{Offset}, \text{Size}\}$$

- **PcodeOp:** La singola operazione elementare del P-Code, definita da un codice operativo (opcode) e dai relativi Varnode, che costituisce l'unità base per l'analisi semantica del binario.

Tecnica di analisi

Metodologia: individuazione e astrazione del flusso logico di WannaCry tramite la scomposizione dei blocchi funzionali.

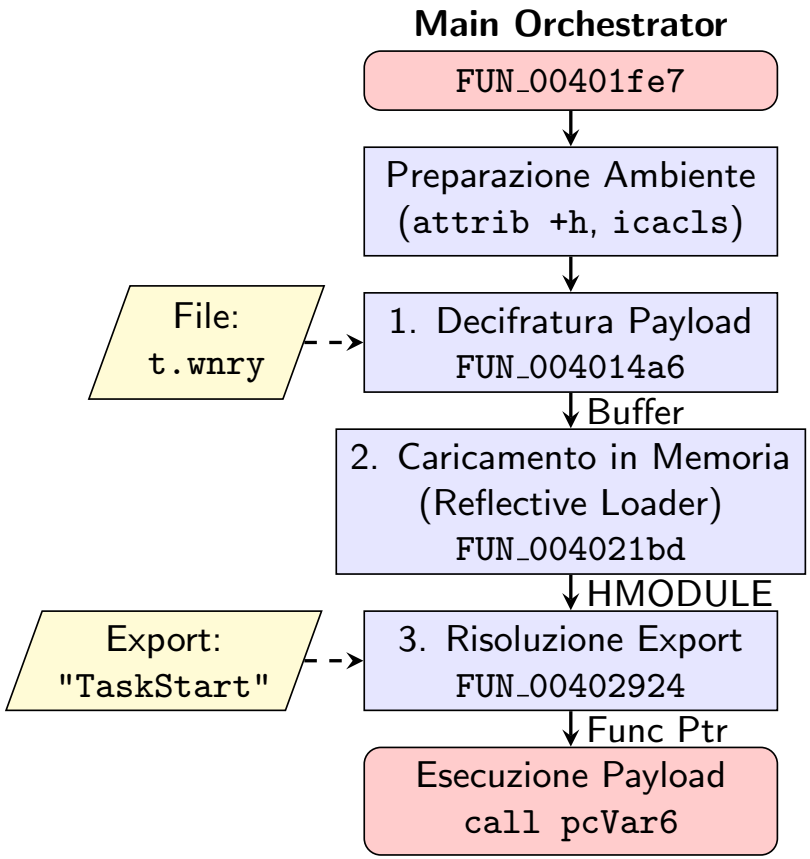


Fig. 1: Flusso dell'Orchestrator

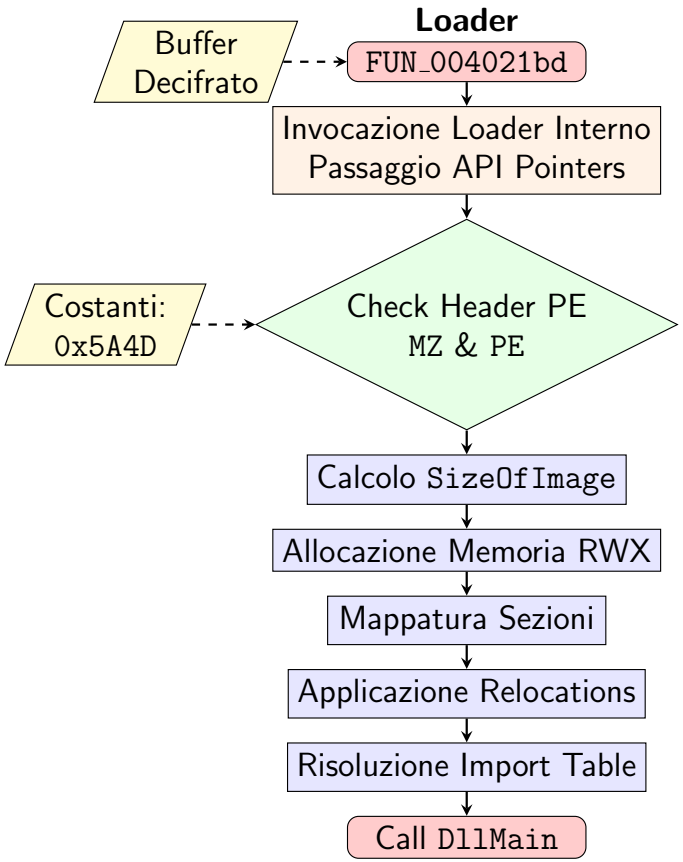


Fig. 2: Flusso del Reflective Loader

Sviluppo dello Script

Obiettivo: Identificazione del **Loader**, il modulo responsabile dell'estrazione e dell'attivazione del payload finale.

1. **Identificazione API**
   - **Dynamic API Resolution:** Analisi delle funzioni dedicate alla risoluzione dinamica degli indirizzi.
   - **Identificazione Wrapper:** Identificazione automatica e ridenominazione dei wrapper.

2. **Trigger**
   - **Punto di ingresso:** La scansione viene innescata a partire dalle chiamate a primitive di decifratura (`CryptDecrypt`)..

3. **Scansione Euristica**
   - **Scansione Semantica:** Ricerca di indicatori critici (flag) quali l'uso di `VirtualAlloc` con permessi `RWX` e individuazione dei *Magic Bytes* (`PE`, `MZ`)

Validazione sul caso di studio (WannaCry)

La verifica è stata condotta sull'eseguibile `tasksche.exe`, cuore della catena di infezione.

Esito dell'Analisi

- **Identificazione dei Wrapper :** Riconoscimento e rinomina di 5 funzioni critiche (*VirtualAlloc, LoadLibrary, etc.*).
- **API Resolution:** Ridenominazione automatica di 13 puntatori (es. `PTR_CryptDecrypt`).
- **Detection:** Identificazione della funzione `FUN_004021e9` come Loader.

Risultato Finale

```
[*] Start Point:  CryptCall in
FUN_00401a45
-> Scanning:  FUN_00401a45
  -> Scanning:  FUN_0040170a
    -> Scanning:  FUN_00401fe7
  [?]  Checking FUN_004021e9:  Score
100
    (Found MZ, PE, Wrappers:  5)
[SUCCESS] LOADER PE IDENTIFICATO:
FUN_004021e9
```

Meccanismo di Fallback

- **Modalità Brute-Force**

Validazione su altri casi

Lo script è stato esteso a campioni con tecniche di evasione differenti per testarne la generalizzazione.

Campione	Caratteristica	Score	Analisi del Risultato
Emotet	Offuscamento API	**80**	**Successo:** Rilevati `MZ` e `PE`.
FormBook	Remote Injection	**0**	**Vero negativo:** Architettura differente.
Cobalt Strike	Reflective Loader	**80**	**Falso Positivo:** Funzioni legittime.

Limiti Emersi

- **Specificità:** Difficoltà nel distinguere tra controllo degli header malevolo e inizializzazione legittima.

Possibili Miglioramenti

- **Liste di esclusione**: Integrazione di database di funzioni legittime per ridurre i falsi positivi.
- **De-offuscamento:** Potenziamento dell'analisi delle API Resolution.

Conclusioni

- **Astrazione tramite P-Code:** L'utilizzo della Rappresentazione Intermedia di Ghidra ha permesso di svincolare l'analisi dalla sintassi dell'architettura, focalizzandosi esclusivamente sulla semantica delle azioni.

- **Generalizzazione dei pattern:** Il focus sulla fase di loading ha messo in luce come l'identificazione di logiche universali possa risultare una strategia più versatile rispetto alla creazione di firme vincolate a specifiche varianti.

- **Efficacia del metodo:** La ricerca di un equilibrio tra sensibilità e specificità suggerisce che la capacità di astrarre i comportamenti rappresenta una strategia valida contro minacce in continua evoluzione.

Grazie per l'attenzione

Il caso WannaCry:
analisi statica con Ghidra e sviluppo sperimentale di uno script di detection

Candidato: Stefano Raglianti